API Specification Doc

“Fábula Dental” Management System

Version	Date	Author	Description
2.0	23-Jun-2026	CodeXplorers Team	URIs list

Methods

Business Logic

http://34.56.164.152:3001

Authentication and Roles:
Before testing any Business Logic server URI, you must log in to obtain a JWT token.
Depending on the role you log in with, you will have access to the following modules:

Rol	Module that you can test
Administrator	Supplies
Dentist	Patients
Receptionist	Payments

How to use a JWT token in Postman?

The token is NOT added manually to the headers.

1. First, log in (see the Auth section below).
2. Copy the token value from the response.
3. In Postman, go to the Authorization tab.
4. In the Type field, select Bearer Token.
5. Paste the token into the Token field.

How to use an API key for CRUD URIs?

The CRUD server URIs (http://136.113.240.33:3000) require an API key.

In Postman:

Go to the Headers tab.

Add the following key and value:

Key	Value
x-api-key	admin

Authentication (Auth)

1. Login

Authenticates a registered user by validating their credentials against the stored bcrypt-hashed password. On success,returns a signed JWT token and the user's role for client-side session management.

Request

Method	URL
POST	/fabuladental/auth/login

You have to send the one of the next fields into the body (raw fi JSON)
{"username": "dentista", "password": "admin"}

{"username": "administrador", "password": "admin"}

{"username": "recepcionista", "password": "admin"}

Response

HTTP code	Response
200	{"message": "Login exitoso.", "token": "", "role": "Dentist"}
500	{"error": "username and password are required."}
401	{"error": "Authentication failure. Invalid credentials."}
500	{"error": "Internal error attempting to authenticate."}

Save the token value — you'll need it in the Authorization → Bearer Token tab of Postman to test the rest of the URIs.

2. Logout

Invalidates an active JWT session by adding the token to a server-side blacklist, preventing further use of that token for authenticated requests.

Request

Method	URL
POST	/fabuladental/auth/logout

You have to send the JWT token in the Authorization header as a Bearer token: Authorization: Bearer

Response

HTTP code	Response
200	{"message": "Session successfully closed."}
400	{"error": "No token found in the Authorization header."}

3. Register a new user

Registers a new system user by hashing the plain-text password with bcrypt before forwarding the request to the CRUD server. Accepts only the roles: Administrator, Dentist, and Receptionist.

Request

Method	URL
POST	/fabuladental/auth/register

You have to send the next fields into the body (raw fi JSON) {"username": "dentista", "password": "admin", "role": "Dentist"}

Response

HTTP code	Response
200	{"success": true, "message": "User registered successfully"}
400	{"error": "Missing required fields or invalid role."}

500	{"error": "Internal error while trying to register the user."}

Supplies

1. Update supplies expiration status

Compares the `expirationDate` of all inventory items against the current system date, automatically calculating and updating the `status` field (`Current`, `NextExpiration`, `Expired`) in the database.

Request

Method	URL
`POST`	`/fabuladental/supplies/status-calculations`

Response

HTTP code	Response
`200`	`{"success": true, "message": "Supplies expiration status successfully updated.", "processedItems": 1}`
`500`	`{"error": "Failed to calculate or update expiration statuses."}`

2. Get inventory total asset valuation

Calculates the total monetary value of the active warehouse stock in real time by multiplying each item's current `quantity` by its registered `unitCost`.

Request

Method	URL
`GET`	`/fabuladental/supplies/asset-valuations`

Response

HTTP code	Response

200	{"totalInventoryValue": 201.03, "currency": "USD", "calculatedAt": "2026-06-09T19:30:00Z"}
500	{"error": "Unable to compile asset valuation data. "}

3. Get restock provisioning and budget analysis

Evaluates all items with a quantity less than or equal to 5, calculates the units needed to reach the clinic's ideal stock of 15 units per item, and sums their estimated costs to deliver a consolidated replenishment budget.

Request

Method	URL
GET	/fabuladental/supplies/restock-provisions

Response

HTTP code	Response
200	{"consolidatedRestockBudget": 14.12, "currency": "USD", "itemsToPurchase": [{"id": 2, "supplyName": "Resina", "unitsToOrder": 14, "estimatedCost": 14}, {"id": 3, "supplyName": "Paracetamol", "unitsToOrder": 12, "estimatedCost": 0.12}]}
500	{"error": "Unable to calculate restock provisioning analysis."}

4. Calculate expired supplies financial loss

Calculates the total direct financial loss of the clinic by summing the total cost (quantity * unitCost) of all inventory items whose status is currently marked as 'Expired'.

Request

Method	URL
GET	/fabuladental/supplies/expiration-losses

Response

HTTP code	Response
200	{"totalLossAmount": 91, "currency": "USD", "expiredItemsCount": 2, "calculatedAt": "2026-06-09T19:30:00Z"}
500	{"error": "Could not calculate historical expiration losses."}

5. Get next-expiration capital at risk assessment

Quantifies the total financial value at risk of being wasted by multiplying the quantity by the unitCost of all items whose current status is marked as 'NextExpiration'.

Request

Method	URL
GET	/fabuladental/supplies/capital-risks

Response

HTTP code	Response
200	{"totalCapitalAtRisk": 0.3, "currency": "USD", "riskPeriod": "30 Days or Less", "itemsNearExpirationCount": 1}
500	{"error": "Failed to compile the expiration capital risk assessment."}

Patients

6. Calculate patient pediatric category

Takes a patient's date of birth as an input, calculates the exact age in years and months, and automatically maps the patient to their respective clinical age group classification for safe dosage validation.

Request

Method	URL
POST	/fabuladental/patients/pediatric-category

You have to send a birthdate with a correct format into the body (choose raw -> JSON)

{"birthday": "15/05/2018"}

Response

HTTP code	Response
200	{"calculatedAgeYears": 8, "calculatedAgeMonths": 0, "pediatricCategory": "School-age", "dosageRestrictionFactor": "0.75x"}
400	{"error": "Valid birthday format required."}
500	{"error": "Internal processing error calculating clinical category."}

7. Validate legal representative

Takes a patient's date of birth and legal representative field as input, calculates their exact age, and enforces the presence of a legal representative if the patient is underage (under 18).

Request

Method	URL
POST	/fabuladental/patients/legal-representative-validation

You have to send the next fields into the body (raw -> JSON)

{"birthday": "12/05/2005", "legalRepresentative": ""}

Response

HTTP code	Response
200	{"valid": true, "requiresLegalRepresentative": false, "message": "Legal representative not required."}
400	{"error": "Valid birthday format required."} {"error": "Patient is underage, legal representative is required."}
500	{"error": "Internal processing error validating legal representative."}

8. Calculate days to birthday

Takes a patient's date of birth as input and calculates how many days are left until their next birthday, and whether their birthday is within the current week (7 days or less).

Request

Method	URL
POST	/fabuladental/patients/days-to-birthday

You have to send the next fields into the body (raw -> JSON)

{"birthday": "15/08/1990"}

Response

HTTP code	Response
200	{"daysUntilBirthday": 63, "isBirthdayWeek": false}
400	{"error": "Valid birthday format required."}
500	{"error": "Internal processing error calculating days to birthday."}

9. Calculate senior discount

Takes a patient's date of birth as input, calculates their exact age, and determines if they are eligible for a senior discount (65 years or older), returning the suggested discount factor.

Request

Method	URL
POST	/fabuladental/patients/senior-discount

You have to send the next fields into the body (raw -> JSON)

{"birthday": "10/01/1954"}

Response

HTTP code	Response
200	{"suggestedDiscountFactor": 0.5, "isEligibleForDiscount": true}
400	{"error": "Valid birthday format required."}
500	{"error": "Internal processing error calculating senior discount."}

10. Estimate consultation time

Takes a patient's date of birth and reason for consultation as inputs. It assigns a base consultation time and dynamically adds minutes if the patient is very young (< 5), elderly (> 70), or if their reason for consultation is complex (lengthy description)..

Request

Method	URL
POST	/fabuladental/patients/consultation-time-estimation

You have to send the next fields into the body (raw -> JSON)

{"birthday": "10/01/2021", "reasonForConsultation": "El paciente presenta un dolor agudo en la muela inferior derecha que no lo deja dormir y tiene inflamación severa en las encías."}

Response

HTTP code	Response
200	`{"estimatedConsultationMinutes": 25, "baseMinutes": 15}`
400	`{"error": "Valid birthday format required."}`
500	`{"error": "Internal processing error estimating consultation time."}`

11. Calculate contact priority

Takes a patient's phone number and reason for consultation to assign a priority score. Higher scores are given if they provide contact means and if their reason includes urgent keywords (e.g., "dolor", "urgencia","sangrado","emergencia").

Request

Method	URL
POST	`/fabuladental/patients/contact-priority`

You have to send the next fields into the body (raw -> JSON)

{"phone": "0991234567", "reasonForConsultation": "Tengo mucho dolor y sangrado por favor es una urgencia."}

Response

HTTP code	Response
200	`{"contactPriorityScore": 60, "requiresImmediateAttention": true}`

500	{"error": "Internal processing error calculating contact priority."}

Payments

12. Get payment history

Retrieves the payment history and calculates current status (Pending, Partial, Completed).

Request

Method	URL
GET	/fabuladental/payments

Response

Http code	Response
200	[{"paymentID": "...", "amount": 50.0, "status": "Partial"}, ...]
500	{"error": "Unable to fetch payment history."}

13. Retrieve a specific payment record

Retrieves the complete details of a particular payment record by its ID, returning the data as a JSON object.

Request

Method	URL
GET	/fabuladental/payments/:paymentId

In the URI you have to send the payment id (like 1, 2, 3….)

/fabuladental/payments/5

Response

Http Code	Response

200	{"success": true, "data": { "id": "5", "patientId": "1727417859", "amount": 50, "type": "Final", "status": "Completed", "date": "2026-05-13T00:00:00.000Z" }
400	{"error": "Payment ID required."}
404	{"error": "Payment not found."}
500	{"error": "Failed to retrieve payment details."}

14. Retrieve all payment records for a specific patient

Query and return the complete history of all payment records associated with a specific patient using their ID. The response is delivered as a JSON object containing a list.

Request

Method	URL
GET	/fabuladental/payments/patients/:patientId

In the URI you have to send the patient id (this is a DNI number)

/fabuladental/payments/patients/1727417859

Response

Http Code	Response
200	{ "success": true, "data": [{ "id": "5", "amount": 50, "type": "Final", "status": "Completed", "date": "2026-05-13T00:00:00.000Z" }, { "id": "11", "amount": 50,

	<pre>"type": "Final",
	"status": "Completed",
	"date": "2026-05-13T00:00:00.000Z"
	}
	]
	}</pre>

400	<pre>{"error": "Invalid Patient ID parameter."}</pre>
404	<pre>{"error": "Patient not found or no payment records discovered."}</pre>
500	<pre>{"error": "Failed to retrieve patient payment history."}</pre>

CRUD Operations

http://136.113.240.33:3000

The Users URIs in CRUD (http://136.113.240.33:3000) are used internally by the Business Logic server to validate credentials and register users.

How to use an API key for CRUD URIs?

The CRUD server URIs (http://136.113.240.33:3000) require an API key.

In Postman:

Go to the Headers tab.

Add the following key and value:

Key	Value
x-api-key	admin

Users

1. Register a new user

Registers a new system user into the database with an encrypted password and an assigned business role.

Request

Method	URL
POST	/fabuladental/users

You have to send the next fields into the body (raw -> JSON)


```
{

  "username": "dentist_andres",

  "password":
"$2b$10$E8eYpYm8bXvR7cK1qL6wOe5mS2z3yG9xM4oUeZ7xO5mUbeHcl8Ym6",

  "role": "Dentist"

}
```


Response

HTTP code	Response

201	{
	  "success": true,
	  "message": "User registered successfully"
	}

400	{ "error": "Missing required fields or invalid role."

	}

500	{
	  "error": "Database error while registering user."
	}

2.Get users by username

A CRUD query that retrieves the full information of a single user identified by their unique username to support the authentication validation process.

Request

Method	URL
GET	/fabuladental/users/:username

In the URI you have to send the username

/fabuladental/users/dentist_andres

Response

HTTP code	Response

200	{
	  "id": 2,
	  "username": "dentist_andres",
	  "password": "$2b$10$E8eYpYm8bXvR7cK1qL6wOe5mS2z3yG9xM4oUeZ7xO5mUbeHcl8Ym6",
	  "role": "Dentist",
	  "created_at": "2026-06-23T19:30:00.000Z",
	  "updated_at": "2026-06-23T19:30:00.000Z"
	}

404	{
	  "error": "User not found."
	}

500	{ "error": "Database error while fetching user."

	}

Supplies

1. Get supplies list

Retrieves the complete list of dental supplies directly from the database, returning all persisted fields matching the database schema.

Request

Method	URL
GET	/fabuladental/supplies

Response

HTTP code	Response

200	[
	  {
	  "id": "2",
	  "supplyName": "Resina",
	  "quantity": 1,
	  "unitCost": "1",
	  "orderDate": "2010-10-10T00:00:00.000Z",
	  "expirationDate": "2020-10-11T00:00:00.000Z",
	  "status": "Expired",
	  "created_at": "2026-05-11T00:44:44.000Z",
	  "updated_at": "2026-05-11T02:10:48.000Z"
	  }, …
	]

500	{"error": "Unable to fetch inventory."}

2.Get supplies filtered by maximum quantity

A CRUD query that reads and filters supplies from the database whose current stock is less than or equal to the maximum quantity specified in the URL path.

Request

Method	URL
GET	/fabuladental/supplies/quantity-thresholds/:maxQuantity

In the URI you have to send the maximum quantity

`/fabuladental/supplies/quantity-thresholds/2`

Response

HTTP code	Response

200	<pre>[
	    {
	        "id": "2",
	        "supplyName": "Resina",
	        "quantity": 1,
	        "unitCost": "1",
	        "orderDate": "2010-10-10T00:00:00.000Z",
	        "expirationDate": "2020-10-11T00:00:00.000Z",
	        "status": "Expired",
	        "created_at": "2026-05-11T00:44:44.000Z",
	        "updated_at": "2026-05-11T02:10:48.000Z"
	    }
	]</pre>

400	{"error": "Invalid maximum quantity format."}
500	{"error": "Database error while filtering supply quantities."}

3.Get supplies filtered by database status

A CRUD query that retrieves records from the database matching the exact status string passed in the URL path (`Expired` or `NextExpiration`)

Request

Method	URL

GET	/fabuladental/supplies/statuses/:statusValue

In the URI you have to send the statusValue (Expired, NextExpiration,Current)

`/fabuladental/supplies/statuses/Expired`

Response

HTTP code	Response

200	<pre>[
	    {
	        "id": "2",
	        "supplyName": "Resina",
	        "quantity": 1,
	        "unitCost": "1",
	        "orderDate": "2010-10-10T00:00:00.000Z",
	        "expirationDate": "2020-10-11T00:00:00.000Z",
	        "status": "Expired",
	        "created_at": "2026-05-11T00:44:44.000Z",
	        "updated_at": "2026-05-11T02:10:48.000Z"
	    },
	    {
	        "id": "1",
	        "supplyName": "Resina",
	        "quantity": 24,
	        "unitCost": "3.75",
	        "orderDate": "2026-05-10T00:00:00.000Z",
	        "expirationDate": "2026-05-10T00:00:00.000Z",
	        "status": "Expired",
	        "created_at": "2026-05-10T20:36:53.000Z",
	        "updated_at": "2026-05-11T02:43:20.000Z"
	    }
	]</pre>

400	<pre>{"error": "Provided status value does not exist."}</pre>
500	<pre>{"error": "Database error while filtering supply statuses."}</pre>

4. Register a dental supply

Registers a new dental supply into the inventory with quantity and dates.

Request

Method	URL
POST	/fabuladental/supply

You have to send the next fields into the body (raw -> JSON)

```
{
  "supplyName": "Resin",
  "quantity": 10,
  "unitCost": 5.50,
  "orderDate": "2026-06-09",
  "expirationDate": "2027-06-09"
}
```

Response

HTTP code	Response
201	{"success": true, "message": "Supply added"}
400	{"error": "Missing supply data."}
500	{"error": "Could not add supply."}

5. Update supply

Edits an existing supply in the inventory by updating any of its modifiable attributes.

Request

Method	URL
PUT	/fabuladental/supplies/:id

In the URI you have to send the supply id (like 1, 2 ,3, …) of the supply you want to update, also you have to send the next fields into the body (raw -> JSON)

/fabuladental/supplies/7

{

"supplyName": "Resin Z350",

"quantity": 15,

"unitCost": 6.25,

"orderDate": "2026-06-09",

"expirationDate": "2027-06-09"

}

Response

HTTP code	Response
200	{"success": true, "message": "Supply updated"}
400	{"error": "Invalid supply ID or quantity."}
404	{"error": "Supply not found."}

6. Delete a dental supply

Removes a dental supply from the inventory using its unique ID.

Request

Method	URL
DELETE	/fabuladental/supplies/:id

In the URI you have to send the supply id (like 1, 2, 3, …)

/fabuladental/supplies/7

Response

HTTP code	Response
200	{"success": true, "message": "Supply deleted"}
400	{"error": "Supply ID required."}
404	{"error": "Supply not found."}
500	{"error": "Deletion failed."}

Payments

7. Get all payments

Retrieves the complete list of all payment records directly from the database. This is the raw data-source endpoint consumed internally by the Business Logic server; it does not compute any derived status and returns the persisted fields exactly as stored.

Request

Method	URL
PUT	/fabuladental/payments

Response

Http Code	Response
200	[{"id": "1", "patientID": "1727417859", "amount": "50.00", "paymentType": "Final", "paymentMethod": "Cash", "status": "Completed", "date": "2026-05-13T00:00:00.000Z", "created_at": "...", "updated_at": "..."}, ...]
500	{"error": "Unable to fetch payment history."}

8. Update payment history

Updates a recorded payment to correct errors or update status.

Request

Method	URL
PUT	/fabuladental/payments/:paymentId

In the URI you have to send the payment id (like 1, 2 ,3, …) of the payment you want to update, also you have to send the next fields into

the body (raw -> JSON)

`/fabuladental/payments/7`

```
{
    "patientID": "1725629313",
    "amount": "45",
    "date": "2020-10-01T00:00:00.000Z",
    "paymentType": "Deposit",
    "paymentMethod": "Transfer",
    "status": "Partial"
}
```

Response

Http Code	Response
200	{"success": true, "message": "Payment updated"}
400	{"error": "Invalid payment ID or data."}
404	{"error": "Payment record not found."}

9. Delete a payment record

Deletes a payment record from history using its id.

Request

Method	URL
DELETE	/fabuladental/payments

You have to send the next fields into the body (raw -> JSON)

{ "id" : 2 }

Response

Http Code	Response
200	{"success": true, "message": "Payment deleted"}
400	{"error": "Payment ID required."}
404	{"error": "Payment not found."}
500	{"error": "Deletion failed."}

10. Record a payment

Records a new patient payment (Deposit or Final).

Request

Method	URL
POST	/fabuladental/payments

You have to send the next fields into the body (raw -> JSON)

{

"patientID": "1727417851",

"amount": "50",

"date": "2026-05-13",

"paymentType": "Final",

"paymentMethod": "Cash",

"status": "Completed"

}

Response

Http code	Response
201	{"success": true, "message": "Payment recorded"}
400	{"error": "Invalid payment data."}
500	{"error": "Could not record payment."}

Patients

11. Register a new patient

Registers a new patient with ID, name, birthday, and reason for consultation.

Request

Method	URL
POST	/fabuladental/patients

You have to send the next fields into the body (raw -> JSON)

```
{
  "patientID": "1727095416",
  "fullName": "Victoria Diaz",
  "phone": "0984284522",
  "email": "cvdiaz3@espe.edu.ec",
  "birthday": "20/03/2003",
  "gender": "Female",
  "reasonForConsultation": "gum pain"
}
```


Response

HTTP code	Response
201	{"success": true, "message": "Patient registered successfully"}
400	{"error": "Missing required fields."}
500	{"error": "Something went wrong. Please try again later."}

12. Get all patients

Retrieves the list of all registered patients to populate the dentist's table dynamically.

Request

Method	URL
GET	/fabuladental/patients

Response

HTTP code	Response
200	[{ "patientID": "1727095415", "created_at": "2026-06-13T02:04:35.964Z",

	<pre>"fullName": "Victoria Diaz",
	"birthday": "2003-03-20T00:00:00.000Z",
	"phone": "0984284522",
	"reasonForConsultation": "gum pain",
	"legalRepresentative": null,
	"gender": "Female"
	}, …
	]</pre>

400	{"error": "Unable to fetch patients."}

13. Get patient by ID

Retrieves the full information of a single patient identified by patientID.

Request

Method	URL
GET	/fabuladental/patients/:patientID

In the URI you have to send the patient id (this is a DNI number)

/fabuladental/patients/1727095414

Response

HTTP code	Response
200	{"patientID": "1727095414", "fullName": "Victoria Diaz", "phone":"0984284522","email":"cvdiaz3@espe.edu.ec","birthday":"20/03/2003","gender":"Female","reasonForConsultation": "gum pain"}
404	{"error": "Patient not found."}
505	{"error": "Unable to fetch patient."}

14. Update patient information

Updates an existing patient's information directly from the table.

Request

Method	URL
PUT	/fabuladental/patients/:patientId

In the URI you have to send the patient id (his DNI number) and you have to send the next fields into the body (raw -> JSON)

```
/fabuladental/patients/1727095414
{
  "patientID": "1727095414",
  "fullName": "Victoria Diaz",
  "phone": "0984284522",
  "email": "cvdiaz3@espe.edu.ec",
  "birthday": "20/03/2003",
  "gender": "Female",
  "reasonForConsultation": "gum pain"
}
```

Response

HTTP code	Response
200	{"success": true, "message": "Patient updated"}
400	{"error": "Patient ID is required."}
404	{"error": "Patient not found."}
500	{"error": "Something went wrong."}

15. Delete a patient

Deletes a patient record.

Request

Method	URL
DELETE	/fabuladental/patients/:patientId

In the URI you have to send the patient id (his DNI number)

/fabuladental/patients/1727095419

Response

HTTP code	Response
200	{"success": true, "message": "Patient deleted"}
400	{"error": "Invalid or missing patient ID."}
404	{"error": "Patient not found."}
500	{"error": "Deletion failed."}

Glossary

Conventions

- **Client** - Client application.
- **Status** - HTTP status code of response.
- All responses are in JSON format except where HTML views are returned.
- All request parameters are mandatory unless marked as [optional].
- All request parameters are mandatory unless explicitly marked as `[optional]`
- The API includes both RESTful endpoints (data operations) and HTML views (forms and dashboards).
- Authentication is required for most endpoints except the public treatments page and login page.

Status Codes

All status codes are standard HTTP status codes. The below ones are used in this API.

`2XX` - Success of some kind
`4XX` - Error occurred in client's part
`5XX` - Error occurred in server's part

Status Code	Description
200	OK
201	Created
400	Bad request
401	Authentication failure
403	Forbidden
404	Resource not found
500	Internal Server Error